

Практическое занятие 30: Настройка прав доступа и ролей пользователей

Цель занятия:

- Настроить права доступа в приложении Django.
 - Ограничить возможность добавления и редактирования мероприятий только администраторам.
 - Разрешить оставлять отзывы только аутентифицированным пользователям.
 - Понять, как использовать декораторы и миксины для контроля доступа.
 - Обновить шаблоны для отображения элементов в зависимости от прав доступа пользователя.
-

Задачи занятия

1. Введение в права доступа и роли в Django
 2. Ограничение доступа к добавлению и редактированию мероприятий
 3. Настройка доступа к добавлению отзывов
 4. Обновление шаблонов в соответствии с правами доступа
 5. Выполнение практического упражнения. Проверка прав доступа
 6. Выполнение дополнительных заданий. Настройка групп и разрешений
-

Задача 1: Введение в права доступа и роли в Django

1.1. Что такое права доступа и роли?

- **Роли пользователей:** Определенные группы пользователей с особыми правами (например, администраторы, редакторы, обычные пользователи).
- **Права доступа:** Разрешения, которые определяют, какие действия может выполнять пользователь в приложении.

1.2. Как Django управляет правами доступа?

- **Декораторы:** Функции, которые можно применять к представлениям для ограничения доступа (например, `@login_required`).

- **Миксины:** Классы, которые можно использовать в классах-представлениях для ограничения доступа (например, `LoginRequiredMixin`).
 - **Система разрешений:** Django предоставляет встроенную систему разрешений на уровне моделей.
-

Задача 2: Ограничение доступа к добавлению и редактированию мероприятий

2.1. Ограничение доступа к представлениям добавления и редактирования мероприятий

Цель: Сделать так, чтобы только администраторы могли добавлять и редактировать мероприятия.

Шаги:

1. **Импортируйте декоратор `user_passes_test` в `views.py`:**

```
# events/views.py

from django.contrib.auth.decorators import user_passes_test
```

2. **Создайте функцию-проверку для определения, является ли пользователь администратором:**

```
# events/views.py

def is_admin(user):
    return user.is_authenticated and user.is_staff
```

3. **Примените декоратор `user_passes_test` к представлениям `add_event` и `edit_event`:**

```
# events/views.py

@user_passes_test(is_admin)
def add_event(request):
    # Ваш существующий код
```

```
# events/views.py

@user_passes_test(is_admin)
```

```
def edit_event(request, event_id):  
    # Ваш существующий код
```

4. Обновите представление удаления мероприятия `EventDeleteView`:

Если вы используете класс-представление, можно использовать `UserPassesTestMixin`:

```
# events/views.py  
  
from django.contrib.auth.mixins import UserPassesTestMixin  
  
class EventDeleteView(UserPassesTestMixin, DeleteView):  
    model = Event  
    template_name = 'events/delete_event.html'  
    success_url = reverse_lazy('events:event_list')  
  
    def test_func(self):  
        return self.request.user.is_authenticated and  
self.request.user.is_staff  
  
    def handle_no_permission(self):  
        return redirect('events:login')
```

Пояснение:

- `test_func`: Метод, который проверяет, имеет ли пользователь необходимые права.
- `handle_no_permission`: Метод, который вызывается, если пользователь не имеет доступа.

2.2. Перенаправление неавторизованных пользователей

- Если неавторизованный пользователь попытается получить доступ к представлению, он будет перенаправлен на страницу входа.

Задача 3: Настройка доступа к добавлению отзывов

3.1. Ограничение доступа к добавлению отзывов только для аутентифицированных пользователей

Цель: Разрешить оставлять отзывы только зарегистрированным и вошедшим в систему пользователям.

Шаги:

1. Импортируйте декоратор `login_required` в `views.py`:

```
# events/views.py

from django.contrib.auth.decorators import login_required
```

2. Обновите представление `event_detail`, чтобы использовать `login_required` для обработки POST запросов:

```
# events/views.py

from django.utils.decorators import method_decorator

@login_required(login_url='events:login')
def post_review(request, event):
    review_form = ReviewForm(request.POST, event=event)
    try:
        if review_form.is_valid():
            review = review_form.save(commit=False)
            review.event = event
            review.user = request.user # Если вы хотите связать отзыв
с пользователем
            review.save()
            messages.success(request, 'Ваш отзыв успешно добавлен.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            messages.error(request, 'Пожалуйста, исправьте ошибки в
форме.')
    except ValidationError as e:
        review_form.add_error(None, e)
    return review_form # Возвращаем форму с ошибками

def event_detail(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    reviews = event.reviews.all().order_by('-created_at')

    if request.method == 'POST':
        # Если пользователь не аутентифицирован, перенаправляем на
страницу входа
        if not request.user.is_authenticated:
            return redirect('events:login')
        # Обрабатываем отзыв
        review_form = post_review(request, event)
    else:
        review_form = ReviewForm(event=event)

    return render(request, 'events/event_detail.html', {
        'event': event,
```

```
        'reviews': reviews,
        'review_form': review_form,
    })
```

Пояснение:

- Разделяем логику обработки отзыва в отдельную функцию `post_review`, которую декорируем `@login_required`.
- Внутри `event_detail` проверяем, если метод `POST` и пользователь не аутентифицирован, перенаправляем на страницу входа.
- Если пользователь аутентифицирован, вызываем `post_review`.

3. Добавьте поле `user` в модель `Review` (опционально):

Если вы хотите связать отзывы с пользователями, можно добавить поле `user` в модель `Review`.

```
# events/models.py

from django.contrib.auth.models import User

class Review(models.Model):
    # ... существующие поля ...
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True,
                             blank=True)
    # ... остальной код ...
```

Не забудьте выполнить миграции:

```
python manage.py makemigrations
python manage.py migrate
```

4. Обновите форму `ReviewForm` (если необходимо):

Если вы добавили поле `user` в модель, убедитесь, что оно не отображается в форме:

```
# events/forms.py

class ReviewForm(forms.ModelForm):
    class Meta:
        model = Review
        fields = ['name', 'email', 'rating', 'comment'] # Не включаем
        поле 'user'
```

Задача 4: Обновление шаблонов в соответствии с правами доступа

4.1. Скрытие кнопок добавления и редактирования мероприятий для неадминистраторов

Шаги:

1. В шаблоне, где находится ссылка на добавление мероприятия (`add_event`), добавьте проверку:

```
<!-- Например, в templates/events/event_list.html -->

{% if user.is_authenticated and user.is_staff %}
    <a href="{% url 'events:add_event' %}">Добавить мероприятие</a>
{% endif %}
```

2. В шаблоне `event_detail.html` скрывайте ссылки на редактирование и удаление для неадминистраторов:

```
<!-- templates/events/event_detail.html -->

{% if user.is_authenticated and user.is_staff %}
    <a href="{% url 'events:edit_event' event.id %}">Редактировать</a>
    <a href="{% url 'events:delete_event' event.id %}">Удалить</a>
{% endif %}
```

4.2. Скрытие формы добавления отзыва для неаутентифицированных пользователей

Шаги:

1. В шаблоне `event_detail.html` оберните форму отзыва проверкой:

```
<!-- templates/events/event_detail.html -->

{% if user.is_authenticated %}
    <!-- Форма добавления отзыва -->
    <h2>Оставить отзыв</h2>
    <form method="post">
        {% csrf_token %}
        {{ review_form.non_field_errors }}
    <div>
        {{ review_form.name.label_tag }}
    </div>
</form>
</div>
```

```
        {{ review_form.name }}
        {{ review_form.name.errors }}
    </div>
    <!-- ... остальные поля формы ... -->
    <button type="submit">Отправить</button>
</form>
{% else %}
    <p><a href="{% url 'events:login' %}?next={{ request.path
}}">Войдите</a>, чтобы оставить отзыв.</p>
{% endif %}
```

Пояснение:

- `{% if user.is_authenticated %}` проверяет, вошел ли пользователь в систему.
- **Предоставляем ссылку на вход**, передавая параметр `next` для перенаправления обратно после входа.

Задача 5: Практическое упражнение: Проверка прав доступа

Задание для студентов:

- **Создайте обычного пользователя** и попробуйте добавить или отредактировать мероприятие.
 - Убедитесь, что у вас нет доступа и вы перенаправлены на страницу входа или получаете сообщение об ошибке.
- **Создайте администратора** (встроенный суперпользователь Django) и убедитесь, что у вас есть доступ к добавлению и редактированию мероприятий.
- **Попробуйте оставить отзыв, не войдя в систему.**
 - Убедитесь, что вы не можете этого сделать и вас перенаправляют на страницу входа.

Цель упражнения:

- Убедиться, что права доступа настроены правильно.
- Понять, как поведение приложения меняется в зависимости от ролей пользователей.

Дополнительные задания

1. Настройка групп и разрешений

Цель: Использовать встроенную систему групп и разрешений Django для более гибкой настройки прав доступа.

Шаги:

1. Создайте группы пользователей в админке Django:

- **Администраторы мероприятий:** Пользователи, которые могут добавлять и редактировать мероприятия.
- **Обычные пользователи:** Пользователи, которые могут оставлять отзывы.

2. Назначьте соответствующие разрешения группам:

- **Администраторы мероприятий:**
 - `events.add_event`
 - `events.change_event`
 - `events.delete_event`
- **Обычные пользователи:**
 - `events.add_review`

3. Обновите функции-проверки в `views.py`, чтобы использовать разрешения:

```
# events/views.py

from django.contrib.auth.decorators import permission_required

@permission_required('events.add_event', login_url='events:login')
def add_event(request):
    # Ваш существующий код
```

Пояснение:

- `@permission_required` проверяет, имеет ли пользователь указанное разрешение.
- Убедитесь, что пользователи имеют соответствующие разрешения через группы.

2. Ограничение редактирования и удаления мероприятий только их создателем

Цель: Разрешить редактирование и удаление мероприятия только пользователю, который его создал (например, администратору).

Шаги:

1. Добавьте поле `created_by` в модель `Event`:


```
# events/models.py

from django.contrib.auth.models import User

class Event(models.Model):
    # ... существующие поля ...
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)
    # ... остальной код ...
```

Не забудьте выполнить миграции.

2. При создании мероприятия сохраняйте информацию о создателе:

```
# events/views.py

@user_passes_test(is_admin)
def add_event(request):
    if request.method == 'POST':
        form = EventForm(request.POST)
        if form.is_valid():
            event = form.save(commit=False)
            event.created_by = request.user
            event.save()
            messages.success(request, 'Мероприятие успешно добавлено.')
            return redirect('events:event_detail', event_id=event.id)
        else:
            form = EventForm()
    return render(request, 'events/add_event.html', {'form': form})
```

3. Обновите представления `edit_event` и `EventDeleteView`, чтобы проверять, является ли пользователь создателем мероприятия:

```
# events/views.py

@user_passes_test(is_admin)
def edit_event(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    if event.created_by != request.user:
        return redirect('events:event_detail', event_id=event.id)
    # Остальной код представления
```

```
# events/views.py

class EventDeleteView(UserPassesTestMixin, DeleteView):
    # ... ваш существующий код ...

    def test_func(self):
```

```
event = self.get_object()
return self.request.user.is_authenticated and self.request.user
== event.created_by
```

Заключение

Поздравляем! Вы успешно:

- Настроили права доступа и роли пользователей в вашем приложении.
- Ограничили доступ к определенным действиям только для администраторов.
- Разрешили оставлять отзывы только аутентифицированным пользователям.
- Научились использовать декораторы и миксины для контроля доступа.
- Обновили шаблоны для отображения элементов в зависимости от прав доступа пользователя.

Если у вас возникнут дополнительные вопросы или потребуется помощь с каким-либо шагом — обращайтесь!

Успехов в разработке!

Вопросы для обсуждения

- Почему важно ограничивать доступ к определенным действиям только для администраторов?
- Ограничение доступа повышает безопасность приложения, предотвращает несанкционированные изменения данных и функций, гарантирует, что только уполномоченные пользователи могут выполнять критические действия, такие как добавление или редактирование мероприятий.
- Как декораторы и миксины помогают в управлении правами доступа в Django?
- Декораторы позволяют легко применять проверку прав доступа к функциональным представлениям, добавляя условия для выполнения действий. Миксины, используемые с классами-представлениями, предоставляют встроенные методы для проверки прав доступа, обеспечивая повторное использование кода и упрощая управление доступом на уровне классов.
- Какие преимущества дает использование встроенной системы разрешений Django при настройке ролей пользователей?

- Встроенная система разрешений Django обеспечивает гибкость и масштабируемость управления доступом, позволяя создавать кастомные разрешения, группировать пользователей и назначать им соответствующие права. Это упрощает администрирование и обеспечивает централизованный контроль над доступом к различным частям приложения.
-

Полезные ресурсы

- [Документация Django: Управление доступом пользователей](#)
- [Документация Django: Декораторы доступа](#)
- [Документация Django: Миксины доступа](#)
- [Видео-туториал по правам доступа в Django](#)